Proxmox Meets Enterprise PKI – Michael Waterman

michaelwaterman.nl/2026/02/18/proxmox-meets-enterprise-pki

Michael Waterman February 18, 2026

Over the past few months, I've been writing extensively about building and configuring Microsoft Enterprise PKI (AD CS). And if you're anything like me, you probably run some kind of home lab where you test ideas before deploying them at work.

At the same time, there's a noticeable shift happening in the industry. Proxmox is showing up more and more, not just in home labs, but increasingly in enterprise environments as well. The same applies to my own setup. I've been running Proxmox for over two years now and I genuinely like the platform.

One thing that has been bothering me for a while, however, is that it uses a self-signed certificate for its management web interface. While the connection itself is still encrypted and protected (*contrary to popular belief*), it simply isn't how things should be in a properly managed environment. So let's fix that.

Let's use the Active Directory Certificate Services infrastructure we've been building over the past few weeks to issue a certificate that is fully trusted by our endpoints. This post goes fairly hardcore, mostly command line, minimal user interface… well, maybe just a little. I'll show you how to:

- generate private keys
- create a certificate request on Proxmox
- request, obtain, and install the certificate
- configure Proxmox to use it properly

And we won't stop there. We'll cover both traditional RSA certificates and modern Elliptic Curve (ECC) certificates, with a clear preference for ECC.

Let's dive in.

Background, how Proxmox handles certificates

Before replacing anything, it helps to understand how Proxmox manages certificates internally. The Proxmox web interface (`pveproxy`) uses a certificate and private key stored in the cluster filesystem under `/etc/pve`. This filesystem is shared across nodes and behaves slightly differently from a regular Linux directory.

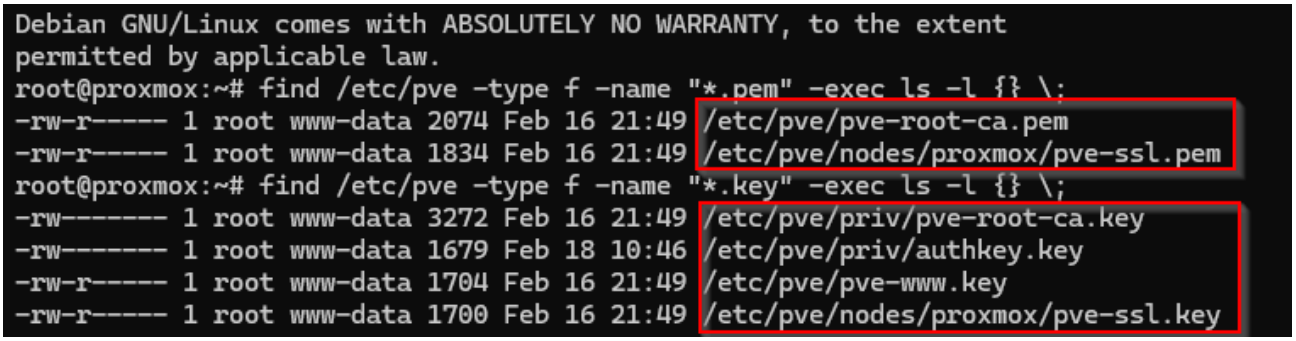
You'll typically find:

- the node certificate

- the private key
- the internal Proxmox CA

To explore them, connect to your Proxmox server via SSH and list the certificate and key files.

```
find /etc/pve -type f -name "*.pem" -exec ls -l {} \;
find /etc/pve -type f -name "*.key" -exec ls -l {} \;
```



You will typically see files like:

- `pve-root-ca.pem`
- `/nodes/<nodename>/pve-ssl.pem`
- `/nodes/<nodename>/pve-ssl.key`

An important detail is that `/etc/pve/local` is effectively a reference to `/etc/pve/nodes/<hostname>`. This is where we will place our new certificate.

Step 1 – Generate the private key

We'll generate a new key that will later be signed by the Windows CA. We cover both approaches:

- ECC (recommended)
- RSA (compatibility option)

RSA vs ECC – Quick decision guide (TL;DR)

When deploying TLS certificates, you'll typically choose between RSA and Elliptic Curve Cryptography (ECC). Both are secure, but they serve different purposes.

ECC (Recommended for most modern deployments)

- Smaller key sizes (256-bit ECC ≈ RSA 3072–4096 security)
- Faster TLS handshakes
- Lower CPU usage on servers
- Better performance at scale

- Modern default for internal services and new deployments

Use ECC unless compatibility requires otherwise.

RSA (Compatibility choice)

- Maximum compatibility with legacy systems
- Required in some enterprise PKI policies
- Widely supported everywhere
- Larger keys and slower performance

Use RSA when legacy clients or policy require it.

Option A – ECC (Recommended)

Modern TLS deployments prefer elliptic curve cryptography because it provides strong security with smaller keys and faster handshakes. And let's face it, our Proxmox server has better things to do than shake large hands (*insider joke*), generate a P-256 key:

```
openssl ecparam -name prime256v1 -genkey -noout \
-out /etc/pve/local/pveproxy-ssl.key
```

Option B – RSA (Traditional)

Use RSA if you need maximum compatibility or your PKI policies require it.

```
openssl genrsa -out /etc/pve/local/pveproxy-ssl.key 4096
```

Step 2 – Create certificate request configuration

We define the certificate properties and "*Subject Alternative Names*" in a configuration file, this will serve as the input for our certificate request file.

ECC configuration

```
[ req ]
default_md         = sha256
distinguished_name = req_distinguished_name
req_extensions     = v3_req
prompt             = no

keyalg = EC
curve  = prime256v1

[ req_distinguished_name ]
C  = NL
ST = Noord-Brabant
```

```
L  = Veldhoven
O  = Corp
OU = IT
CN = proxmox.corp.michaelwaterman.nl

[ v3_req ]
keyUsage = critical, digitalSignature
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = proxmox.corp.michaelwaterman.nl
DNS.2 = proxmox
```

Note! The "*alt_names*" do not need to be the same as the actual host name", choose a name that makes sense to you. Save as:

```
/etc/pve/local/pveproxy-ssl.conf
```

Tip! verify your host name with: `hostname -f`

RSA configuration (alternative)

```
[ req ]
default_bits       = 4096
distinguished_name = req_distinguished_name
req_extensions     = v3_req
prompt             = no

[ req_distinguished_name ]
C  = NL
ST = Noord-Brabant
L  = Veldhoven
O  = Corp
OU = IT
CN = proxmox.corp.michaelwaterman.nl

[ v3_req ]
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = proxmox.corp.michaelwaterman.nl
DNS.2 = proxmox
```

Step 3 – Generate the certificate signing request (CSR)

Next up is the creation of the certificate request, later on we'll use this request to ask our Enterprise CA if it would please create the certificate for us. Here are both options.

ECC CSR

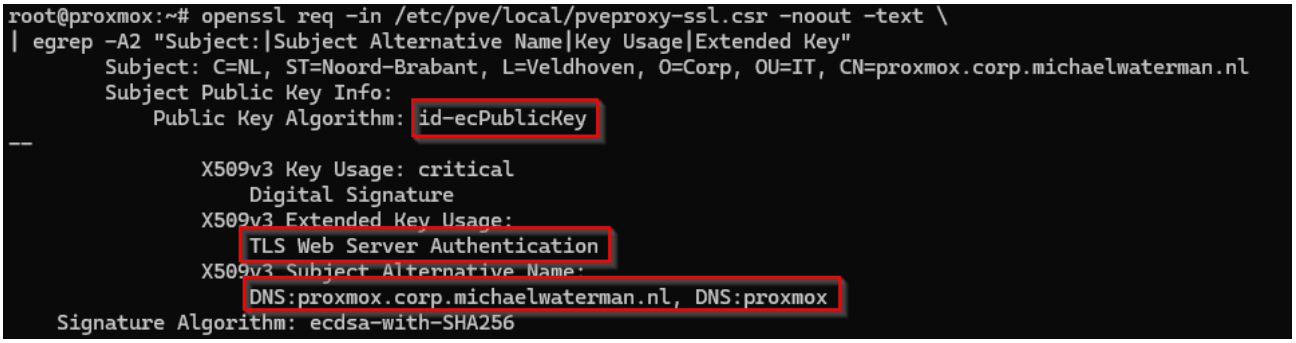
```
openssl req -new -newkey ec \
-pkeyopt ec_paramgen_curve:prime256v1 \
-nodes \
-keyout /etc/pve/local/pveproxy-ssl.key \
-out /etc/pve/local/pveproxy-ssl.csr \
-config /etc/pve/local/pveproxy-ssl.conf
```

RSA CSR

```
openssl req -new \
-key /etc/pve/local/pveproxy-ssl.key \
-out /etc/pve/local/pveproxy-ssl.csr \
-config /etc/pve/local/pveproxy-ssl.conf
```

Now before we want to submit the request, let's validate it first. Lucky for us there's a neat trick in openssl that can do that for us.

```
openssl req -in /etc/pve/local/pveproxy-ssl.csr -noout -text \
| egrep -A2 "Subject:|Subject Alternative Name|Key Usage|Extended Key"
```



Step 4 – Request certificate from Windows CA

Copy the CSR contents to a Windows machine by using the following command on your Proxmox SSH session, select the output and copy it to a file on your Windows management machine.

```
cat /etc/pve/local/pveproxy-ssl.csr
```

I'm going to store the content in a file named "pveproxy-ssl.csr" in a directory "*Certs*" on my desktop (*yeah I know*).

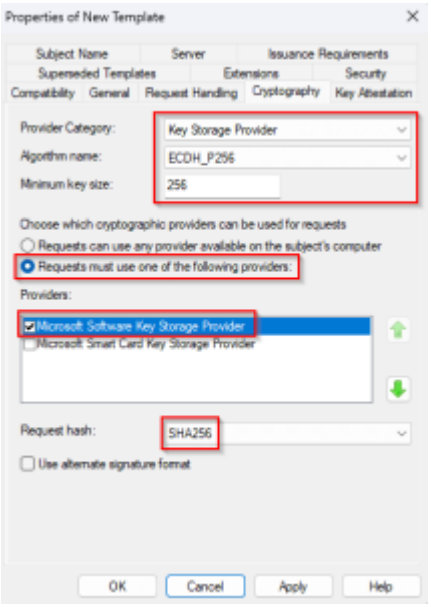
Certificate template configuration

I wanted to use a certificate template I made previously for my Windows Admin Center (*more on that in the future*). It's really a standard template for server authentication, but

there's a difference when you want to use ECC. The regular template is suited for compatibility, meaning it supports RSA, which is fine, but we want to use ECC specifically. So let's make that possible!

Open up your PKI management console, browse to the "*Certificate Templates*", right click and choose "*Manage*". Open the template that uses "*Server Authentication*" and open the "*Cryptography*" tab. Change the defaults to:

- Provider Category: *Key Storage Provider*
- Algorithm name: *ECDH_P256*
- Minimum key size: *256*
- Request hash: *SHA256*



Leave the rest default. Open the "*Subject Name*" tab and change the following:

- Select, "*Supply in the request*"
- Optionally you can select "*Use subject information from existing certificates…*", but for our case it has no effect.

Last, but certainly not least, and this is actually important if not the most important setting, enable the "***CA certificate manager approval***". Without this setting, combined with settings on the "*Subject Name*" tab you will be compromised in minutes. Enable it!

Obtaining the certificate

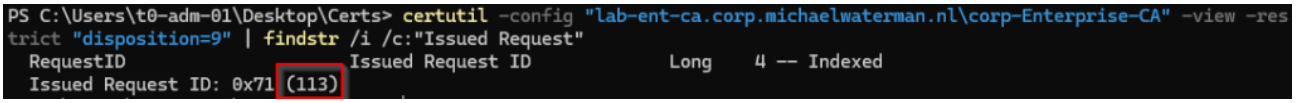
Open a terminal Window in the directory where you stored the "*pveproxy-ssl.csr*" file. Submit the request to AD CS:

```
certreq -config "lab-ent-ca.corp.michaelwaterman.nl\corp-Enterprise-CA" `
-submit -attrib "CertificateTemplate:TLSServerAuthentication" pveproxy-ssl.csr
```

Tip! If you need to locate the name of your CA server, use:

After submitting the request, remember the "*Request ID*" that's shown on the screen. You'll need it the next few commands. But if you get distracted and need to find the ID again, I'll show you a neat a little trick, use the following command to see your pending certificate requests on the CA:

```
certutil -config "lab-ent-ca.corp.michaelwaterman.nl\corp-Enterprise-CA" -view -
restrict "disposition=9" | findstr /i /c:"Issued Request"
```



As you can see, my request is number "113". I'll use the ID in the commands below. Just in case you want to know more on the "*disposition*" parameter, here's what I know of the function.

Disposition	Explanation
9	Pending Certificates
20	Issued Certificates
21	Revoked Certificates
30	Failed Certificates

Anyways, I'm digressing, we have submitted the request and we now need to approve it. Assuming you're authorized to issue certificates, run the following command:

```
certutil -config "lab-ent-ca.corp.michaelwaterman.nl\corp-Enterprise-CA" -resubmit
<ID>
```

Retrieve the certificate:

```
certreq -retrieve -config "lab-ent-ca.corp.michaelwaterman.nl\corp-Enterprise-CA"
<ID> pveproxy-ssl.cer
```

Note! The certificate is already Base64 encoded, no conversion required.

Step 5 – Build the certificate chain

Proxmox needs the certificate chain to function correctly, starting with the leaf certificate, followed by the intermediate and root ca. Technically you could get away with just the root ca certificate, but for completeness I'll show you how to include both.

Download the root and intermediate certificates.

```
Invoke-WebRequest http://trust.corp.michaelwaterman.nl/crl/Corp-Root-CA.crt -
OutFile Corp-Root-CA.crt
Invoke-WebRequest http://trust.corp.michaelwaterman.nl/crl/Corp-Enterprise-CA.crt -
OutFile Corp-Enterprise-CA.crt
```

We need to convert to PEM (Base64) as the certificate chain can't be in binary format (DER):

```
certutil -encode Corp-Root-CA.crt Corp-Root-CA.pem
certutil -encode Corp-Enterprise-CA.crt Corp-Enterprise-CA.pem
```

Now it's time to create the full certificate chain, it's not really that exciting, just a file with 3 certificates in it. Create the chain:

```
Get-Content pveproxy-ssl.cer, Corp-Enterprise-CA.pem, Corp-Root-CA.pem |
Set-Content pve-certificate-chain.pem
```

Step 6 – Install the certificate on Proxmox

At this point you could open your Proxmox web based management interface, browse to the node you're targeting, open "*System*" -> "*Certificates*" and add the certificate information there. I did promise in the beginning of this blog that we would go hardcore command-line, so for today I'm not going to show you this. If you have reached this point, I assume you're quite capable of figuring that out by yourself. Instead SSH into your Proxmox server again and create the following file:

```
nano /etc/pve/local/pveproxy-ssl.pem
```

Copy the content of the "*pve-certificate-chain.pem*" to this file.

Tip! Use the following on your Windows management machine to copy the content to the clipboard.

```
Get-Content-Path.\pve-certificate-chain.pem | Set-Clipboard
```

Verify certificate and key match

We've come a long way to create the private key and certificate, and we want to make sure that we didn't mess things up. Let's see if we can check if the certificate actually matches the previously created private key. There's an option I use within openssl that can generate a hash value of the private key and the public certificate. If those two match we can be sure that we've correctly created the certificate. I'll show you for both ECC and RSA methods

ECC validation (Recommended method)

```
openssl pkey -in /etc/pve/local/pveproxy-ssl.key -pubout -outform DER | openssl sha256
openssl x509 -in /etc/pve/local/pveproxy-ssl.pem -pubkey -noout | openssl pkey -pubin -outform DER | openssl sha256
```

```
root@proxmox:~# openssl pkey -in /etc/pve/local/pveproxy-ssl.key -pubout -outform DER | openssl sha256
openssl x509 -in /etc/pve/local/pveproxy-ssl.pem -pubkey -noout | openssl pkey -pubin -outform DER | openssl sha256
SHA2-256(stdin)= 7b1f69848a1f9782d298483ca3623ca8b61347b86b297189fb346efa0a9f6ef3
SHA2-256(stdin)= 7b1f69848a1f9782d298483ca3623ca8b61347b86b297189fb346efa0a9f6ef3
root@proxmox:~#
```

As you can see, the hashes match, so we can proceed with the installation. But first let me show you the RSA way.

RSA validation

```
openssl rsa -noout -modulus -in pveproxy-ssl.key | openssl sha256
openssl x509 -noout -modulus -in pveproxy-ssl.pem | openssl sha256
```

Assign the certificate

The last step, assigning the certificate and restarting the services. Exciting isn't it!

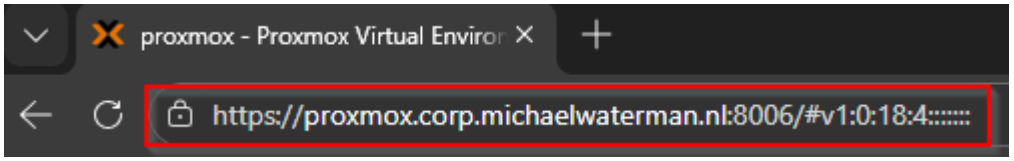
```
pvenode cert set /etc/pve/local/pveproxy-ssl.pem /etc/pve/local/pveproxy-ssl.key -force
systemctl restart pveproxy pvedaemon
```

If all goes well, you will see output similar to this.

```
root@proxmox:~# pvenode cert set /etc/pve/local/pveproxy-ssl.pem /etc/pve/local/pveproxy-ssl.key -force
Setting custom certificate files
```

filename	pveproxy-ssl.pem
fingerprint	7E:35:2F:29:21:73:39:DE:00:8F:1E:76:A0:C1:6F:D6:97:60:F0:BF:6B:AF:2B:E7:56:C4:2F:76:60:5A:80:6A
subject	/C=NL/ST=Noord-Brabant/L=Veldhoven/O=Corp/OU=IT/CN=proxmox.corp.michaelwaterman.nl
issuer	/DC=nl/DC=michaelwaterman/DC=corp/CN=Corp-Enterprise-CA
notbefore	2026-02-18 11:34:26
notafter	2026-03-18 11:34:26
public-key-type	id-ecPublicKey
public-key-bits	256
san	- proxmox.corp.michaelwaterman.nl - proxmox

All that's left now is to open your browser, make sure the public certificate of the Root CA is in the trusted authorities store and browse to the location of the Proxmox server.



No more annoying certificate warnings, Enjoy!

Troubleshooting (Quick reset)

Just in case you need to quickly go back and reset (as I have done many times during the creation of this blog, sigh…), use the following commands to restore connectivity:

```
rm -f /etc/pve/local/pveproxy-ssl.pem /etc/pve/local/pveproxy-ssl.key
pvecm updatecerts --force
systemctl restart pveproxy
```

This restores default certificates.

Closing thoughts

With that, your Proxmox interface is now backed by a certificate issued from your own enterprise PKI. No more browser warnings, centralized lifecycle management, and a setup that actually fits into a modern security architecture.

For internal environments, certificates with a one-year validity are usually perfectly fine. You control the trust, the renewal process, and the lifecycle, and shorter lifetimes can even be a good thing from a security perspective.

Sure, automated solutions like ACME can make certificate management easier. But ACME isn't always an option, especially in isolated networks or tightly controlled enterprise environments. In those cases, integrating Proxmox with your Windows CA gives you a fully controlled and reliable alternative.

Whether you choose RSA for compatibility or ECC for modern performance, your Proxmox deployment now follows the same trust model as the rest of your infrastructure. And let's be honest, after all that time building your PKI, it's nice to finally give it an actual job.

Until next time!